

Beskrivelse av valgt løsning

Vi gikk for en nyskaping i vårt valg av klient og server. Det handler om Blazor, en storsatsing fra Microsoft. Blazor er i spydspissen av webutvikling, og vi måtte basere oss på at Blazor er relativt upløyd mark. Første utgave kom i 2018 (<https://en.wikipedia.org/wiki/Blazor>) og siste i 2020. Nokså kort tid for et rammeverk med ambisjonsnivået Microsoft har lagt seg på. Vi tok høyde for dette ved å gjennomgå et 15 timer langt kurs før vi startet oppgaven (<https://www.udemy.com/course/blazor-the-complete-guide/>).

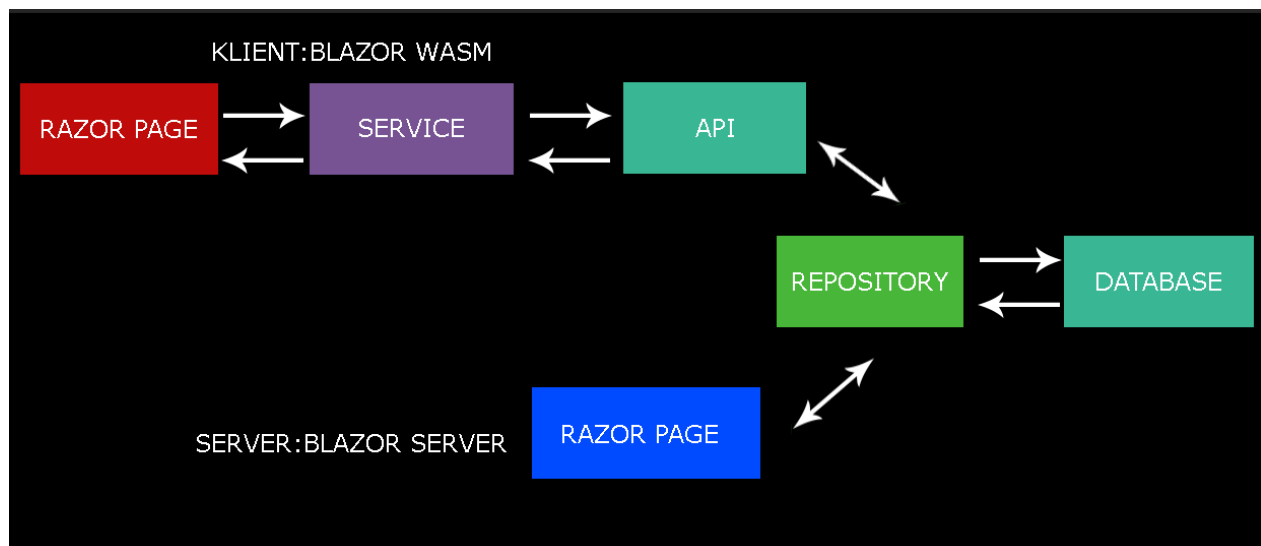
Blazor server for serverside og Blazor WebAssembly(Wasm) for klientside.

Det er mange likheter mellom modellene men også noen forskjeller. I vertsmodellen (på server) kjører Blazor innenfra en ASP.NET core app. Alle oppdateringer med hensyn til brukergrensesnitt, hendelseshåndtering og javascript kall er håndtert over SignalR rett ut av boksen.

På klientsiden blir Blazor WebAssembly (Blazor app med avhengigheter og .NET runtime) lastet ned til browseren, og appen kjører direkte på UI tråden til browseren. All grafisk grensesnitt og hendelseshåndtering skjer innenfor samme prosess. Dette fører noen ganger til litt lavere oppdateringsfrekvens enn man er vant med, og vi anbefaler ikke å bruke eldre browsere.

Kort fortalt betyr dette for vårt prosjekt at WASM i første ledd henvender seg til browseren, mens serveren bruker SignalR og henvender seg til databasen uten andre mellomledd enn repository.

I vår app er Blazor Server og Blazor Wasm på to forskjellige linjer Logistikken eller kommunikasjonslinjene mot database er lagt opp på følgende vis



Klient og server er i all hovedsak isolert fra hverandre, men har indirekte kontakt gjennom sentral database på kark. De deler også samme repository, men utover det avskåret på det viset at de bruker forskjellige måter å kontakte DB på. Årsaken er at klienten bruker http for å kommunisere med øvrig infrastruktur. Server bruker SignalR, socket-kommunikasjon.

Det går ingen direkte linje mellom Klient og server på tegningen. men det finnes ett unntak. I noen tilfeller der hastighet er av betydning og serveren styrer alle forespørsler, som i chat, er det hensiktsmessig at server og klient har direkte kontakt. På serveren er det gjerne en hub, som håndterer innkommende og utgående meldinger fra klienten, og tar seg av arbeidet/logikken, mens klient-side fungerer som en tynnklient, mer som et vindu.

I forhold til DB har vi valgt å bruke DTOer(Data transfer object) som arbeidsmodeller (i prosjektet models). Det er hensiktsmessig å skille mellom arbeidsmodeller og entiteter fordi vi kan manipulere DTO uten å øve innflytelse på databasen. DTO oversettes til entiteter i siste bindeledd, repositoriene, mot databasen, utover dette benytter vi ikke entiteter i kode. Oversettelsen skjer automatisk ved bruk av Automapper.

I prosjektet har vi brukt DTO for å simulere logikk og algoritmer, som et alternativ til seeding. Dette har den fordelen at alt skjer i kode uavhengig av database. DTOene kan man manipulere uten at det har utilsiktede konsekvenser. Det tar følgelig litt lengre tid å sette opp enn å seede, men man slipper å tenke på migrasjon og oppdatering av databasen.

Kort beskrivelse av hvert delprosjekt og formålet med de tjener:

Common: Inneholder SD, som står for standard details. Her havner strenger som beskriver detaljer rundt roller, JWT og annet som midlertidig befinner seg i local storage

API: kontrollere. Deriblant AccountController som tar seg av logikk rundt roller, registrering, login og logout av brukere

Server: kontroll av utført arbeid. Forbehold Admin og Seniorer, med unntak av en testchat vi prøvde på server tidligere). Administrasjon av alle brukere.

Shared: ukjent status, var tiltenkt Chat

Klient: Frontend for freelancere og kunder. Registrerer seg som brukere, men divergerer på tilgang etter rolle

Business: Repository, oversetting av DTO til entitet. Logikk. kommunikasjon mot db. Alltid Siste ledd før db.

Crypto: En konsoll-app for rask overføring mellom kontoer. Her kan man kopiere navnet (eng: label) isteden for adresser for å beskrive hvem man skal sende fra og til.

Models: Her er arbeidsmodellene. Dette er de vi bruker overalt i koden, bortsett fra repository, som oversetter Dto til entitet.

DataAccess: Alt relatert til database. Entiteter, Seeding og entiteter.

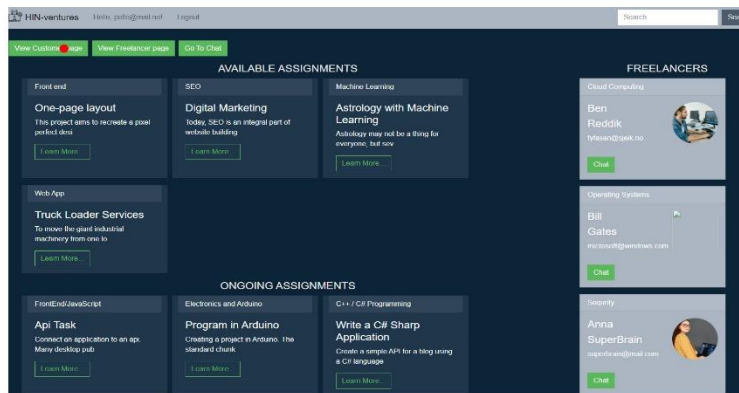
Vi har i stor grad fulgt samme mappestruktur som i tidligere nevnte kurs.

Programflyt klient

Hovesiden på klienten er tilgjengelig og åpen for anonyme brukere, men for å gå videre til sider knyttet til spesifikke roller er det nødvendig å registrere seg. Under registrering kan brukeren velge hvilken rolle han tilhører. Dette skjer via AuthenticationService som sender en forespørsel med innfylte data (UserRequestDto), om å registrere brukeren, til AccountControlleren. Hvis registreringen går bra returneres en RegistrationResponseDto

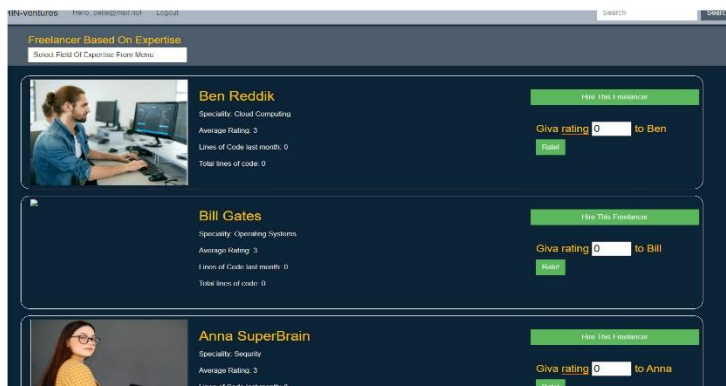
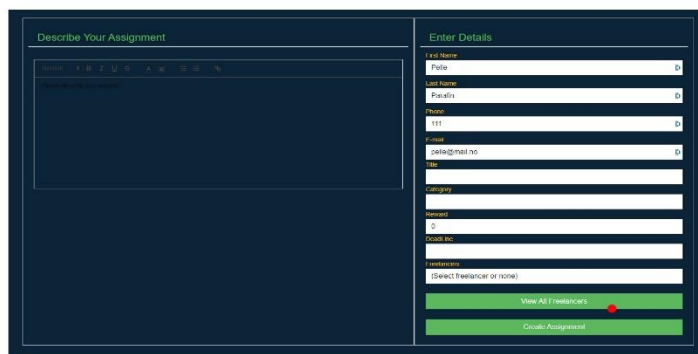
Etter registrering blir brukeren sendt til innlogging og kan logge inn med sin nye bruker. Første side etter login er hovedsida som alle har tilgang til. Hvor brukeren beveger seg videre avhenger av hvilken rolle som ble valgt ved registrering. Her er en grafisk fremstilling for en Customer, starter fra forsiden. Rød prikk representerer klikk.

1 Kunde



Forsiden viser tilgjengelige oppdrag og oppdrag som allerede er tatt av andre frilansere, men som også kan bli ledige for senere tidspunkt. Til høyre har man en liten oversikt over frilansere og med tilgang til chat for hver enkelt av dem.

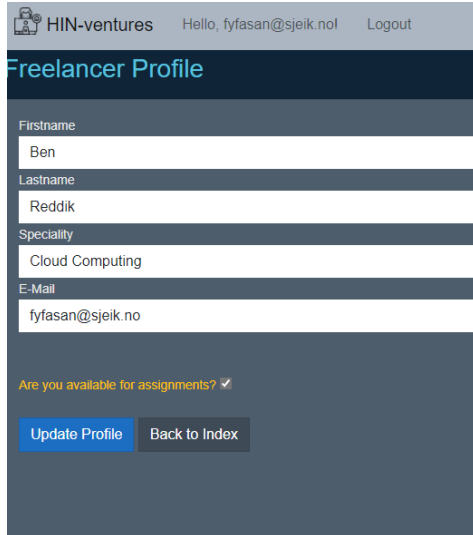
Når kunden går inn på kundesida har han til å opprette assignment. Her fyller han ut en beskrivelse og øvrige detaljer. Noen detaljer er allerede fylt ut. Disse detaljene kommer fra local storage. De lagres der for hver gang en bruker logger inn, og slettes når han logger ut. JWT tokens er brukt gjennom alle sidene for autorisasjon. Kunden kan også velge om han vil la være å velge frilanser tilknyttet oppdraget. I så måte blir oppdraget gjort tilgjengelig for alle frilansere.



Kunden kan også velge frilanser fra en nedtrekksliste, eller gå inn på en oversikt over alle frilansere. Her kan kunden se detaljer vedr. alle frilansere, og sortere etter spesialfelt ved å velge fra en nedtrekksliste. Ved å trykke på 'hire this freelancer' sendes kunden tilbake til forrige side med navn utfyllt i nedtrekkslista

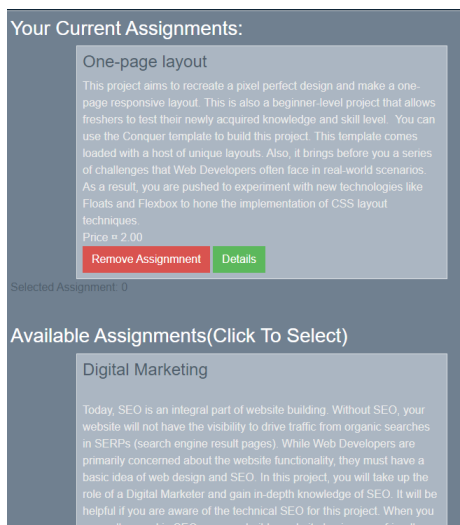
I oversikten over alle frilansere kan også kunden gi en rating til frilanseren , dersom frilanseren har levert et oppdrag (i skrivende stund vites det ikke status for kontroll av ferdig levert oppdrag, akkurat nå kan kundene gi rating når som helst).

2 Frilanser

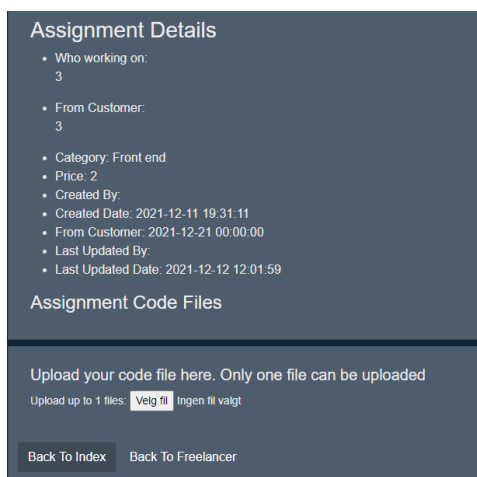


Første side for frilanser er todelt. Først kan vedkommende oppdatere detaljer om seg selv. Alle felter er ferdig fylt inn, samme som på customer, bortsett fra Spesialitet.

I tillegg kan frilanseren velge å sette seg selv tilgjengelig eller utilgjengelig. I så fall kan ikke kunden velge frilanseren (flagget blir satt i databasen når frilanseren trykker oppdater, men det vites ikke i skrivende stund om funksjonen er tatt hensyn til på kundesiden).



Under oppdatering av profil vises kunden oppdrag han har meldt seg på og tilgjengelige oppdrag. Kunden kan bare klikke på tilgjengelige oppdrag så havner de øverst og det er live oppdatering i databasen, slik at oppdraget blir gjort utilgjengelig for alle andre frilansere automatisk. Samme gjelder andre veien. Frilanseren kan melde seg av og oppdraget blir tilgjengelig for alle andre frilansere.



Dette er siden som dukker opp når frilanseren klikker 'details' på et oppdrag. Her kan frilanseren se flere detaljer rundt oppdraget og laste opp kodefil(er).

Programflyt Server

På serveren har admin eksklusiv tilgang.


Log in

Email

admin@gmail.com

Password

☐ Remember me?

Log in

[Forgot your password?](#)

[Register as a new user](#)

[Resend email confirmation](#)

Home

Manage Assignments

Manage Freelancers

Manage Customers

Fetch data

Manage Transactions

Chat room

Hello, wor

Welcome to your new app.

How is Blazor work

```
Enter which user(label) you would like to transfer from :
Pelle_Parafin
Enter which user(label) you would like to transfer to :
HIN-Ventures
Enter Amount you would like to transfer(and press enter):
20
New Balance for HIN-Ventures: {
  "network": "DOGETEST",
  "available_balance": "906.93616000",
  "pending_received_balance": "0.00000000",
  "balances": [
    {
      "user_id": 2,
      "label": "HIN-Ventures",
      "address": "2N1spDd7mDaBfHbCRKGq9p28BwBo2XjmrEF",
      "available_balance": "906.93616000",
      "pending_received_balance": "0.00000000"
    }
  ]
}
New Balance for Pelle_Parafin{
  "network": "DOGETEST",
  "available_balance": "139.99146000",
  "pending_received_balance": "0.00000000",
  "balances": [
    {
      "user_id": 5,
      "label": "Pelle_Parafin",
      "address": "2MzEN1W2fGnr1eGqkkAUxZrQxKBZMuLaJPd",
      "available_balance": "139.99146000",
      "pending_received_balance": "0.00000000"
    }
  ]
}
```

Det aller meste er selvforklarende, og vi trenger ikke å gå i dybden på hver side for å forstå hvordan siden fungerer.

Alle valg fra menyen som det står 'Manage' på, leder til såkalte CRUD sider. Det er vanlige operasjoner for Admin. Sidene lister opp alle Assignments, Freelancers og Customers. Her kan admin gå inn på enkelte Assignments, Freelancers eller Customers og redigere, slette eller oppdatere. I tillegg har Admin også mulighet til å opprette Assignments, Freelancers og Customers. De andre elementene i menyen stammer fra uferdige deler av prosjektet, planlagte elementer, eller de fungerer ikke fordi kode som ikke fungerte ble fjernet.

Betalingsløsningen endte opp med å bli et løstrevet konsollprosjekt. Langt fra ideelt, men det får jobben gjort. Oppgaven var også å vise at vi kunne kommuniser med eksternt api. Overføring mellom kontoer skjer i tre steg. Vi kan vise at vi får til steg en av kommunikasjonen. Dette kan man gjøre ved å sette Server som startup prosjekt, og sette breakpoint på CreateTransaction i Services/CryptoService. Kjør i debug modus og velge Manage Transactions fra menyen. Da kan man lese ut at man får kode 200 success fra metoden der breakpoint ble satt. Men det er ikke nok å 'prepare transaction'. Man også må 'create transaction' og 'submit transaction'. I forhold til JSON er dette veldig dårlig beskrevet på hjemmesiden til block.io, og vi ble nødt til å ty til nødløsninger, som det faktisk finnes dokumentasjon på.

```

3 references
public async Task<CryptoDto> CreateTransaction(CryptoDto transaction) transaction = (CryptoDto)
{
    var content = JsonConvert.SerializeObject(transaction); content = "{\"api_key\":\"9da8-f106-e0c7-e733\",\"from_labels\":\"HIN-Ventures\",\"to_labels\":\"Ben_Reddik\",...\"
    var bodyContent = new StringContent(content, Encoding.UTF8, \"application/json\"); bodyContent = (StringContent)
    var response = await client.PostAsync(\"https://block.io/api/v2/prepare_transaction/\", bodyContent); response = (StatusCode: 200, ReasonPhrase: 'OK', Version: 1.1, Content: System.Net.Http
    String res = response.Content.ReadAsStringAsync().Result; s 587ms elapsed res = null, response = (StatusCode: 200, ReasonPhrase: 'OK', Version: 1.1, Content: System.Net.Http.HttpConnectionResponseC...)

    if (response.IsSuccessStatusCode)
    {

```

Kommentarer knyttet til egen løsning

Det finnes en del løs kode. Det vil si kode som vi ikke bruker i prosjektet, men som er levert. BookingDetails er f.eks. en entitet/tabell i db som vi ikke bruker, men ble benyttet i utviklingen inntil vi fant en bedre løsning. Siden BookingDetails inneholder fremmednøkler og cascade-delete kan medføre at flere ting enn ønskelig slettes i databasen har vi valgt å la den ligge i fred siden det begynner å bli knapt med tid.

I tillegg er det kataloger og en del filer som går på Chat-funksjonalitet, men det er uvisst i skrivende stund om den kommer til å virke.

Pr. nå er det også mulig å registrere seg i rolle som både freelancer og customer på klientside for vi syntes det var logisk. Det ene utelukker ikke det andre. Det skaper det litt trøbbel om man registrer seg dobbelt. Brukeren blir utestengt fra alle sider knyttet til rolle, og vi har ikke hatt tid til å skrive nødvendig kode for å løse problemet.

Når det er sagt, så synes vi Blazor er en utrolig fin platform å utvikle på. Vi slipper å forholde oss til mange forskjellige rammeverk for å kommunisere med web og markup. I Blazor er det lagt meget godt til rette for interaktivitet mellom komponenter på websider og kode-logikk, og man kan gjøre all kode i C#. Det betyr at det kjøres kompilert kode isteden for script, og det har mye å si for hastigheten. I tillegg er Blazor Server rask. Alt går via SignalR i utgangspunktet, ferdig oppsatt. Vi har lært mye, om callbacks og autorisasjon for nevne noe.

Etterhvert skjønte vi også at vi hadde laget en del overflødige relasjoner i databasen. Det går på looping. F.eks. en frilanser har en liste med ratings som har en frilanser som har en liste med ratings. Med Linq kan man løse dette enklere. Da holder det å ha en entitet rating som inneholder rating, freelancerid og kunde id. Da kan man hente ut alt man trenger fra den entiteten. Lærte fordelene med Linq kan man si.

Vi synes flyten på websidene fugerer greit, er intuitive og er brukervennelige. Vi er meget fornøyd med hvordan interaksjonen mellom assignments, freelancere og kunder fungerer. Personlig, for min sin del, tror jeg ikke det er siste gang jeg bruker Blazor.